Module- 2

DYNAMIC AND FUNCTIONAL MODELING

Module 2

Dynamic modeling: Events and States, Operations, Nested state diagrams, Concurrency, Advanced Dynamic Modeling Concepts, A sample Dynamic Model, Relationship of Object and Dynamic models.

Functional modeling: Functional models, Data Flow Diagrams, Specifying Operations, Constraints, A sample Functional Model

AGGREGATION CONCURRENCY

- A dynamic model describes a set of concurrent objects.
- A state of an entire system can't be represented by a single state in a single object.
- It is the product of the states of all objects in it.
- A state diagram for an assembly is a collection of state diagram, one for each component.
- By using aggregation we can represent concurrency.
- Aggregation is the "and-relationship"
- Aggregate state corresponds to combined state of all component diagrams

AGGREGATION CONCURRENCY

- Eg: the state of a Car as an aggregation of component states: the Ignition, Transmission, Accelerator, and Brake. The car will not start unless transmission is in neutral. This is shown by the guard expression *Transmission in Neutral* on the transition from Ignition-Off to Ignition-Starting.
- Each component state also has states.
- The state of the car includes one substate from each component.
- Each component undergoes transitions in parallel with all others.

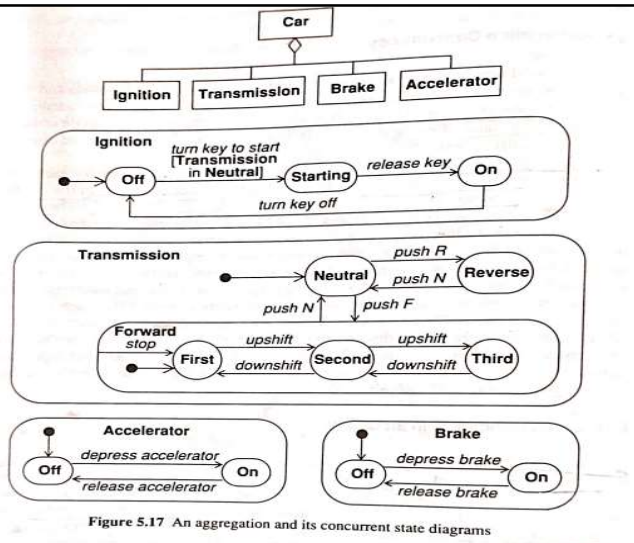


Figure 5.17 An aggregation and its concurrent state diagrams

Concurrency within an Object

- Concurrency within a single composite state of an object is shown by partitioning the composite state into subdiagrams with dotted lines.
- The state of the object comprises one state from each subdiagram.
- The sub diagrams need not be independent; the same event can cause transitions in more than one subdiagram.
- The name of the overall composite state can be written in a separate region of the box, separated by a solid line from the concurrent subdiagrams.

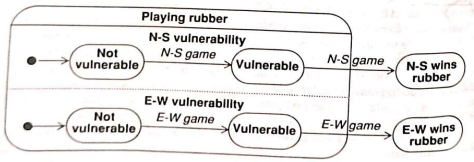


Figure 5.18 Bridge game with concurrent states

- Above figure shows the state diagram for the play of a bridge rubber
- When a side wins a game, it becomes “vulnerable"; the first side to win two games wins the rubber.
- During the play of the rubber, the state of the rubber consists of one state from each subdiagram.
- When the *Playing rubber* composite state is entered, both subdiagrams are initially in their respective default states *Not vulnerable*.
- Each subdiagram can independently advance to state *Vulnerable* when its side wins a game. When one side wins a second game, a transition occurs to the corresponding *Wins rubber* state.
- This transition terminates both concurrent subdiagrams because they are part of the same composite state *Playing rubber* and are only active when the top-level state diagram is in that state.

Advanced Dynamic Modeling Concepts

Entry and Exit Actions

- As an alternative to showing actions on transitions, actions can be associated with entering or exiting a state.
- The action on the incoming transition will be executed first, then the entry action, followed by the activity within the state, followed by the exit action and finally the action on the outgoing transition.

Advanced Dynamic Modeling Concepts

Entry action

- An entry action is a specific action or set of actions that are executed when a system or entity enters a particular state.
- These actions are associated with the state itself and define what should happen as soon as the system transitions into that state.
- An entry action is shown inside the state box following the keyword entry and a "/" character.

Advanced Dynamic Modeling Concepts

Exit action

- An exit action is an action or set of actions that are executed when a system or entity exits a particular state.
- These actions are also associated with the state and define what should happen just before the system leaves that state.
- Exit actions are less common
- An exit action is shown inside the state box following the keyword exit and a "/" character.

- The activities can be interrupted by events causing transitions out of state but entry and exit can't be interrupted.
- The entry/exit actions are useful in state diagrams because they permit a state to be expressed in terms of matched entry-exit actions.

- Figure shows the control of a garage door opener.
- The user generates *depress* events with a push button to open and close the door.
- Each event reverses the direction of the door, but for safety the door must open fully before it can be closed.
- The control generates *motor up* and *motor down* activities for the motor.
- The motor generates *door open* and *door closed* events when the motion has been completed.
- Both transitions entering state *Opening* cause the door to open.

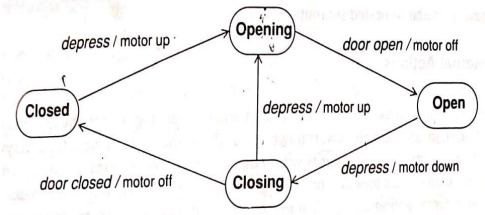


Figure 5.19 Actions on transitions

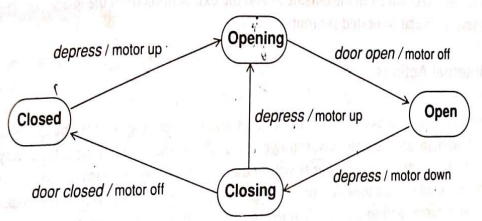
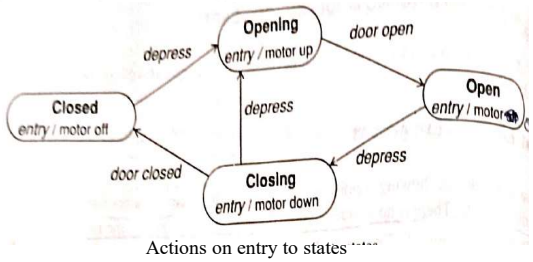


Figure 5.19 Actions on transitions



Actions on entry to states

Advanced Dynamic Modeling Concepts
Internal Action

•An event can cause an action to be performed without causing a state change .

•An internal action refers to an action or event that occurs within a state without the need for external stimuli or transitions.

•The event name is written inside state box followed by "/" and name of action.

Advanced Dynamic Modeling Concepts
Automatic transition

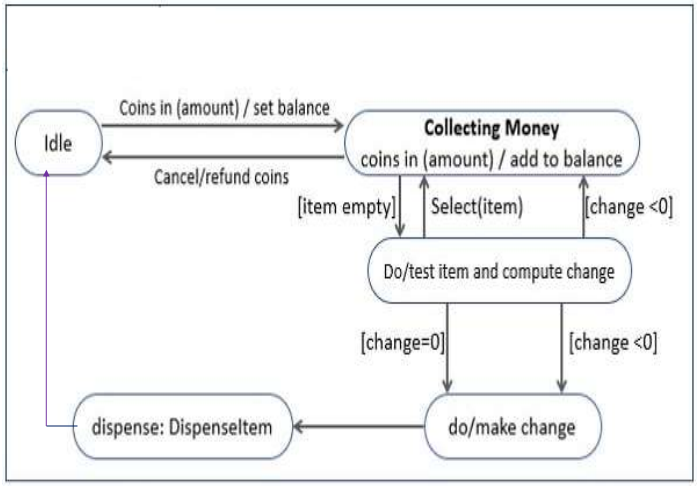
• When an activity is completed , a transition to another state fires.

• An arrow without an event name indicates an automatic transition that fires when the activity associated with the source state is completed.

• If there is no activity, the unlabeled transition fires as soon as the state is entered.

• Such unlabeled transitions are called lambda transitions.

Vending Machine State diagram



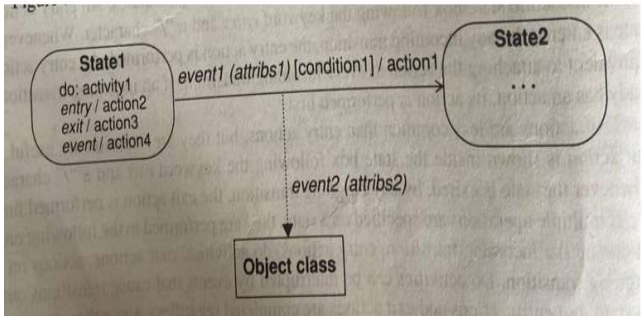
Advanced Dynamic Modeling Concepts

Sending Events

- An object can perform the action of sending an event to another object.
- A system of objects interacts by exchanging events.
- An event can be directed to a single or a group of objects.
- The action "send E(attributes)" send event E with the given attributes to the object or objects that receive it.
- If a state can accept events from more than one object, the order in which concurrent events are received may affect final state.
- This is called a race condition.
- Unwanted race conditions should be avoided.

- Below figure shows another notation for sending an event from one object to another.

- The dotted line from a transition to an object indicates that an event is sent to the object when transition fires.



Synchronization of Concurrent Activities

- Sometimes an object must perform two or more activities concurrently.
- Both activities must be completed before the object can progress to its next state.
- The target state occurs after both events happen in any order.
- Concurrent activities within a single composite activity are shown by partitioning a state into regions with dotted lines.
- Each region is a subdiagram that represents a concurrent activity within the composite activity.
- The composite activity assumes exactly one state from each subdiagram.

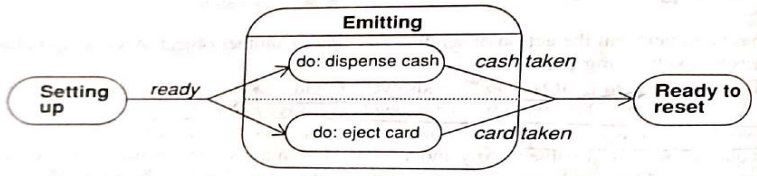
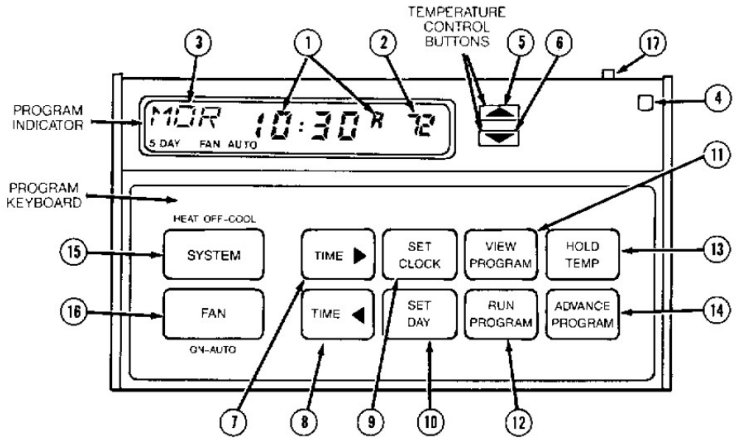


Figure 5.22 Synchronization of control

Sample Dynamic model

PROGRAMMABLE THERMOSTAT



Sample Dynamic model-PROGRAMMABLE THERMOSTAT

- This device controls a furnace and air conditioner according to time-dependent attributes which the owner enters using a pad of buttons.
- While running, the thermostat operates the furnace or air conditioner to keep the current temperature equal to the target temperature.
- The target temperature is taken from a table of program values supplied by the user.
- The table specifies target temperature for 8 different time periods, 4 on weekdays and 4 on weekends, with start times specified by the user.
- The target temperature is reset from the table at the beginning of each program period

- .The user can override-the target temperature for the remainder of the current period or indefinitely.
- The user programs the thermostat using a pad of 10 push buttons and 3 switches.
- The user sees parameters on an alphanumeric display.
- A switch illuminates a night light.
- The thermostat has a temperature sensor that reads the air temperature.
- The thermostat operates power relays for a furnace and an air conditioner, and an indicator lights up when the furnace or air conditioner is operating.

Each push button generates an event every time it is pushed. We assign one input event per button:

TEMP UP	raises target temperature or program temperature
TEMP DOWN	lowers target temperature or program temperature
TIME FWD	advances clock time or program time
TIME BACK	retards clock time or program time
SET CLOCK	sets current time of day
SET DAY	sets current day of the week
RUN PRGM	leaves setup or program mode and runs the program
VIEW PRGM	enters program mode to examine and modify 8 program time and program temperature settings
HOLD TEMP	holds current target temperature in spite of the program
F-C BUTTON	alternates temperature display between Fahrenheit and Celsius

Switches and their settings

Light switch	Lights the alphanumeric display. Values: light off, light on.
Season switch	Specifies which device the thermostat controls. Values: heat (furnace), cool (air conditioner), off (none).
Fan switch	Specifies when the ventilation fan operates. Values: fan on (fan runs continuously), fan auto (fan runs only when furnace or air conditioner is operating).

- The thermostat controls the furnace, air conditioner, and fan power relays.
- We model this control by activities "run furnace," "run air conditioner," and "run fan."
- The thermostat has a temperature sensor that it reads continuously, which we model by external parameter *temp*.
- The thermostat also has an internal clock that it reads and discontinuously.
- We model the clock as another external parameter *time*, since we are not interested in building a state model of the clock.
- We introduce an internal state variable *target temp*, which represents current temperature that the thermostat is trying to maintain and it permits communication among parts of the dynamic model.

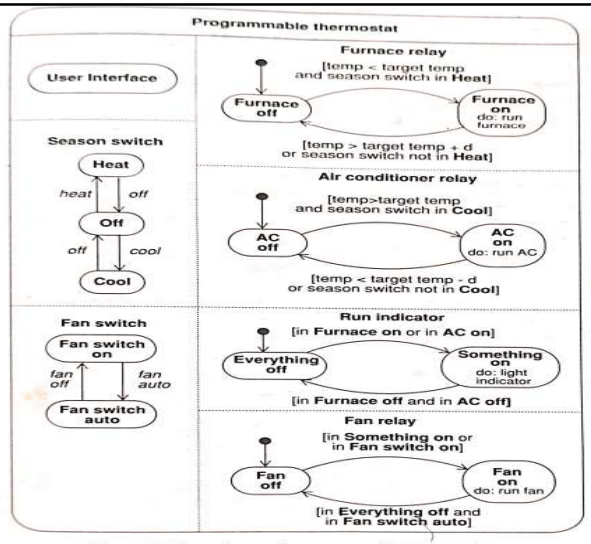


Figure 5.23 State diagram for programmable thermostat

- Top level state diagram has 7 concurrent subdiagrams
- Right side 4 subdiagrams shows output of thermostat
  - Each has on/ off substates
  - UI has 3 concurrent subdiagrams
    - Interactive display,temperature mode ,night light

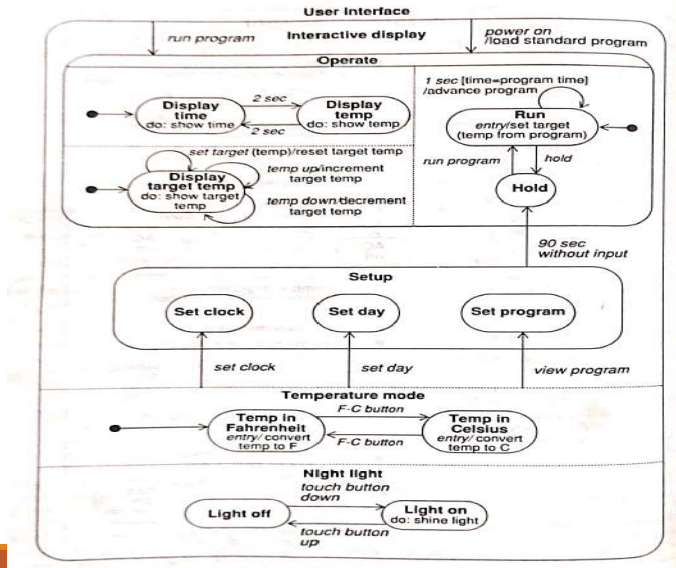


Figure 5.24 Subdiagram for thermostat user interface

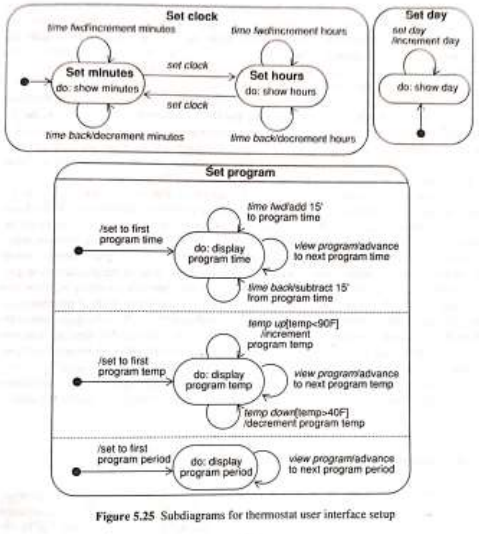


Figure 5.25 Subdiagrams for thermostat user interface setup

Relationship of Object and Dynamic models

- Dynamic model specifies the allowable sequences of changes to objects from the object model
- Dynamic model structure is related to and constrained by object model structure.
- A hierarchy of states of an object is equivalent to a restriction hierarchy of the object class.
- Obiect oriented models and languages do not usually support restriction in the generalization hierarchy, so the dynamic model is the proper place to represent it.
- The dynamic model of a class is inherited by its subclasses.
- States are defined by the interaction of objects and events.
- Transitions can be implemented as operations on objects
- Operation name corresponds to event name

Functional modeling

- The functional model specifies the results of a computation without specifying how or when they are computed.
- The functional model specifies the meaning of the operations in the object model and the actions in the dynamic model
- A spreadsheet is a kind of functional model.
  - In most cases, the values in the spreadsheet are trivial and cannot be structured further.
  - The purpose of the spreadsheet is to specify values in terms of other values.
- A compiler is almost a pure computation.
  - The input is the text of a program in a particular language
  - Output is an object file that implements the program in another language

Data Flow Diagrams(DFD)

- The functional model consists of multiple data flow diagrams.
- A data flow diagram shows the functional relationships of the values computed by a system, including input values, output values, and internal data stores.
- DFD is a graph showing the flow of data values from their sources in objects through processes that transform them to their destinations in other objects.
- DFD does not show control information
- DFD contains processes that transform data, data flows that move data,actor objects that produce and consume data, and data store objects that store data passively.

Processes

- A process transforms data values.
- The lowest level processes are pure functions without side effects.
- An entire data flow graph is a high level process.
- A process may have side effects if it contains nonfunctional components
- A process is drawn as an ellipse containing a description of the transformation(name)
- Each process has a fixed number of input and output data arrows, each of which carries a value of a given type
- The inputs and outputs can be labeled to show their role in the computation, but often the type of value on the data flow is sufficient.
- Processes are implemented as methods of operations on object classes.

Processes

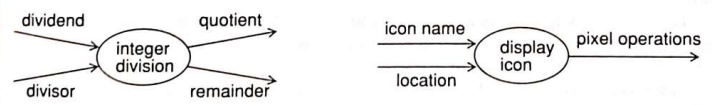
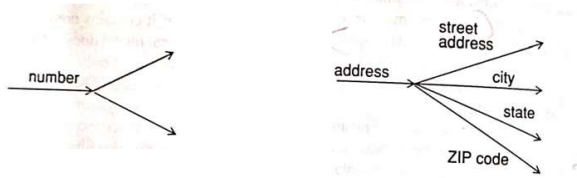


Figure 6.2 Processes

Dataflows

- A data flow connects the output of an object or process to the input of another object or process.
- It represents an intermediate data value within a computation.
- The value is not changed by the flow.
- A data flow is drawn as an arrow between the producer and the consumer of the data value.
- The arrow is labelled with a description of the data(name or type.)
- A fork with several arrows emerging from it indicates that the same value can be sent to several places.
- The output arrows are unlabeled because they represent the same value as the input



Data flows to copy a value and split an aggregate value

Actors

- An actor is an active object that drives the data flow graph by producing or consuming values.
- Actors are attached to the inputs and outputs of a data flow graph.
- The actors lie on the boundary of the data flow graph.
- An actor is drawn as a rectangle

Datastore

- A data store is a passive object within a data flow diagram that stores data for later access.
- A data store is drawn as a pair of parallel lines containing the name of the store.
- Input arrows indicate information or operations that modify the stored data.
- Output arrows indicate information retrieved from the store.
- Both actors and data stores are objects.

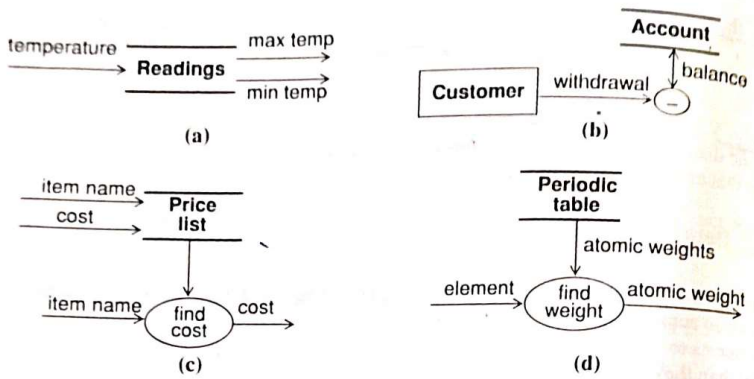


Figure 6.4 Data stores

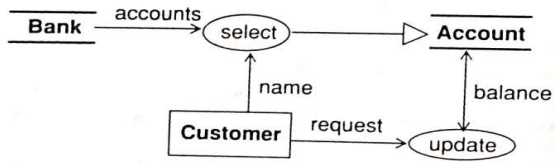


Figure 6.5 Selection with an object as result

- A data flow that generates an object used as the target of another operation is indicated by a hollow triangle at the end of the data flow.
- The update operation modifies the balance in the account object indicated by the small arrowhead.

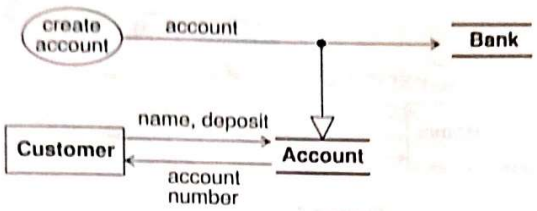


Figure 6.6 Creation of a new object

- The result of the *create account* process is a new account which is stored in the bank.
- The customer's name and deposit are stored in the account.
- The account number from the new account is given to the customer

Nested Data Flow Diagrams

- A process can be expanded into another data flow diagram.
- Each input and output of the process is an input or output of the new diagram.
- The new diagram may have data stores.
- Diagrams can be nested to an arbitrary depth, and the entire set of nested diagrams forms a tree.
- A diagram that references itself represents a recursive computation

Control Flows

- A control flow is a Boolean value that affects whether a process is evaluated.
- The control flow is not an input value to the process itself.
- A control flow is shown by a dotted line from a process producing a Boolean value to the process being controlled.

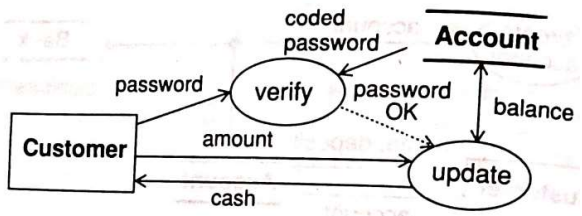


Figure 6.7 Control flow

Specifying Operations

- Each operation can be specified in various ways. These are :
  - Mathematical functions, such as trigonometric functions
  - Table of input and output values for small finite sets
  - Equations specifying output in terms of input
  - Pre- and post- conditions
  - Decision tables
  - Pseudocode
  - Natural language

- Specification of an operation includes a signature and a transformation.
- The external specification of an operation only describes changes visible outside the operation.
- Access operations are operations that read or write attributes or links of an object.
- Nontrivial operations can be divided into three categories: queries, actions, and activities.
  - A query is an operation that has no side effects, it is a pure function.
  - An action is a transformation that has side effects on the target object.
  - An activity is an operation to or by an object that has duration in time

Constraints

- A constraint shows the relationship between two objects at the same time or between different values of the same object at different times.
- A constraint may be expressed as a total function or a partial function.
- Object constraints specify that some objects depend entirely or partially on other objects.
- Dynamic constraints specify relationships among the states or events of different objects.
- Functional constraints specify restrictions on operations.

A sample Functional Model

Flight simulator

- The simulator is responsible for
  - handling pilot input controls
  - computing the motion of the airplane
  - computing and displaying the outside view from the cockpit window
  - displaying the cockpit gauges
- The simulator is intended to be an accurate but simplified model of flying an airplane, ignoring some of the smaller effects and making some simplifying assumptions.

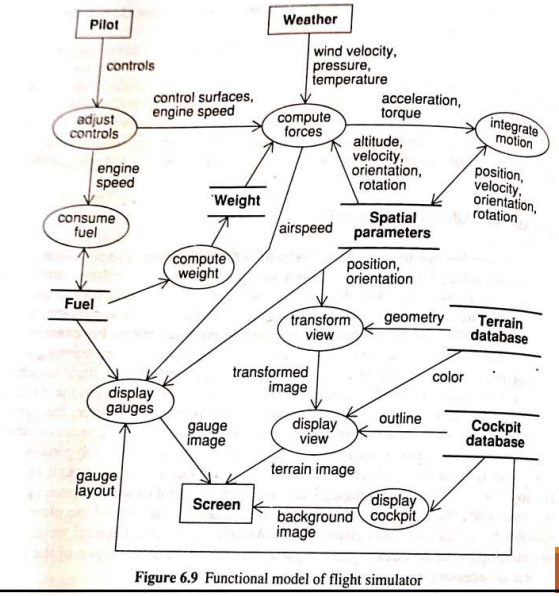


Figure 6.9 Functional model of flight simulator

- The top-level data flow diagram for the flight simulator shows two input actors:
  - the *Pilot*, who operates the airplane controls
  - the *Weather*, which varies according to some specified pattern.
- There is one output actor: the *Screen*, which displays the pilot's view.
- There are two read-only data stores:
  - the *Terrain database*, which specifies the geometry of the surrounding terrain as a set of colored polygonal surfaces
  - the *Cockpit database*, which specifies the shape and location of the cockpit viewport and the locations of the various gauges.

- There are three internal data stores:
  - Spatial parameters , which holds the 3-D position, velocity, orientation, and rotation of the plane;
  - Fuel, which holds the amount of fuel remaining
  - Weight, which holds the total weight of the lane (consumption of fuel causes the weight to decrease).
  - The initialization of the internal stores is necessary but is not shown on the data flow diagram.

- The processes in the diagram can be divided into three kinds:
  - handling controls
  - motion computation
  - display generation.

- **The control handling processes are**
  - adjust controls, which transforms the position of the pilot's controls (joysticks) into positions of the air-plane control surfaces and engine speed;
    - Process adjust control is comprising of three distinct controls:
      - the elevator
      - the ailerons,
      - the throttle.
  - consume fuel, which computes fuel consumption as a function of engine speed;
  - compute weight, which computes the weight of the air-plane as the sum of the base weight and the weight of the remaining fuel.

- **The motion computation processes are**
  - *compute forces*, which computes the various forces and torques on the plane and sums them to determine the net acceleration and the rotational torques
    - incorporates both geometrical and aeronautical computations
  - *integrate motion*, which integrates the differential equations of motion.

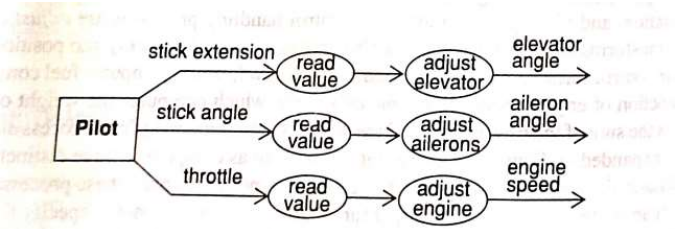


Figure 6.10 Expansion of *adjust controls* process

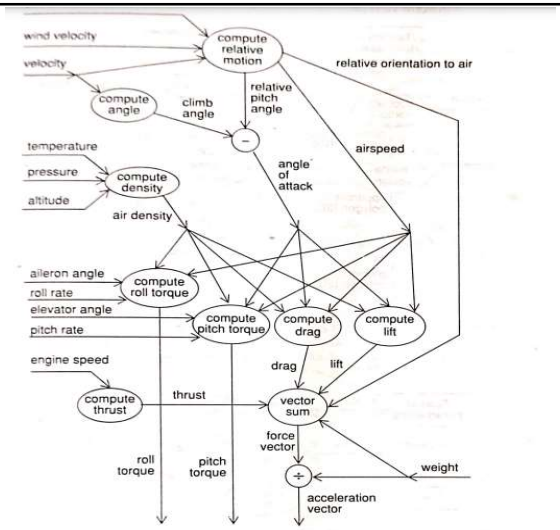
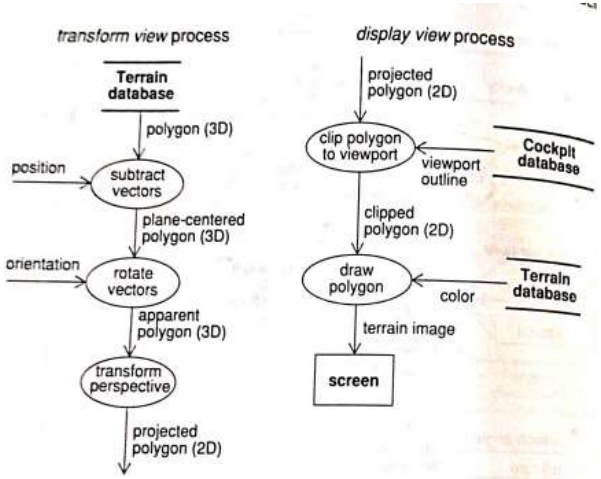


Figure 6.11 Expansion of *compute forces* processes



- The display processes are
  - transform view
  - display view
  - display gauges
  - display cockpit
- These processes convert the airplane parameters and terrain into a simulated view on the screen

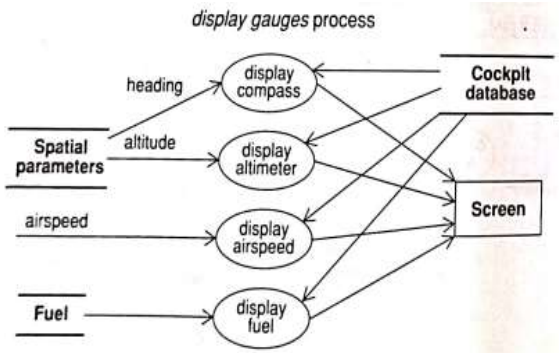


Figure 6.12 Expansion of display processes